FULL-STACK MERN & MONGODB ATLAS CAPSTONE

SmartInventory

Real-Time Warehouse & Stock Management Platform

A modern internal management system featuring atomic multi-document transactions, zero-negative-stock protection, real-time threshold monitoring, and complete audit logging.

PROJECT EVALUATION

Candidate Name:
Shubham Kumar

Registration No.:
12315449

Degree & Branch:
B.Tech Computer Science & Engg.

Department:
School of Computer Science & Engg.

Institution:
Lovely Professional University

Course:
MongoDB Database Design Mini-Project

Academic Year:
2025 – 2026

Project Overview & Real-World Motivation

The Core Mission

Most growing businesses face a critical dilemma: manual spreadsheets (Excel/Google Sheets) cause costly stockouts, while enterprise ERPs (like SAP or Oracle) cost tens of thousands of dollars and require months of training.

SmartInventory provides the ideal sweet spot: a fast, intuitive, full-stack internal management portal that eliminates phantom stock, enforces atomic inventory updates, and automates reorder alerts without software bloat.

01

Eliminating Costly Phantom Inventory

When stock records on paper or spreadsheets disagree with physical warehouse shelves, shipments are delayed and customers leave. SmartInventory guarantees 100% record accuracy.

02

Instant Operational Transparency

Storekeepers and managers get a unified view of warehouse counts, supplier details, low-stock warnings, and historical logs across any device in real time.

03

Auditable Accountability & Compliance

Every single item entering or leaving the warehouse is permanently stamped with the authenticated user ID, transaction timestamp, and reference note.

TALKING POINT FOR PRESENTATION: "Begin by explaining why small-to-medium businesses struggle today. They outgrow Excel sheets because multiple staff edit them without controls, but they cannot afford multi-million dollar ERPs. SmartInventory solves this exact practical problem."

The 4 Core Breakdown Points in Traditional Warehousing

01 Concurrency Chaos

What Happens: When two warehouse clerks log dispatches simultaneously in shared spreadsheets, rows overwrite silently without locking.

Business Impact: Leads to negative stock balances and phantom inventory on physical shelves.

02 Zero Audit Trail

What Happens: Standard spreadsheets allow any staff member to modify or delete numbers with zero history of who touched it or why.

Business Impact: Internal shrinkage, theft, and delivery discrepancies become completely untraceable.

03 Stockout Surprises

What Happens: Managers have no automated warnings when fast-moving items reach dangerous depletion thresholds.

Business Impact: Orders get accepted but cannot be fulfilled, causing lost customer trust and delayed dispatches.

04 Data Corruption

What Happens: Hard-deleting a discontinued product in traditional tables permanently breaks past billing records.

Business Impact: Orphans historical transaction rows and corrupts financial year-end tax calculations.

TALKING POINT FOR PRESENTATION: "Walk the evaluation panel across these 4 cards. Explain that concurrency overwrites cause phantom stock, lack of logs invites theft, missing threshold alerts cause emergency delays, and hard deletes break tax audits."

The SmartInventory Solution Architecture

01 Real-Time Multi-Warehouse Tracking

Provides an instantaneous, unified stock ledger across multiple storage locations with live quantity updates, eliminating manual spreadsheet consolidation.

02 Zero-Negative-Stock Guard

A two-tier validation barrier (React UI check + atomic MongoDB backend conditional update) guarantees that dispatches can never exceed physical on-hand quantity.



03 Immutable Audit Journal

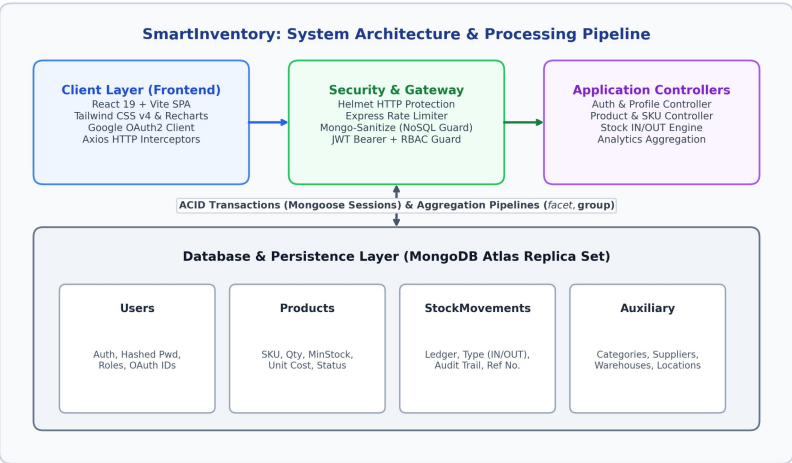
Every single physical movement (Stock IN, OUT, Transfer) automatically logs an unalterable history record tied to the logged-in user, SKU, timestamp, and notes.

04 Automated Threshold Alerts & KPIs

The dashboard dynamically flags products falling below minimum safety limits with visual color badges, while calculating total warehouse valuation instantly.

TALKING POINT FOR PRESENTATION: "Point to the central hub and the 4 satellites. Say: 'SmartInventory acts as the warehouse brain: it tracks stock in real time, strictly blocks overselling, maintains an unalterable audit log, and alerts managers before stockouts occur.'"

Three-Tier Architecture & Technology Stack



Decoupled 3-Tier Flow:
Client Browser (React 19) ⟷ Secure JWT REST API (Express) ⟷ Cloud Cluster (MongoDB Atlas)

01

Frontend Presentation Tier (React 19 + Vite)

Built with React 19, Tailwind CSS, and Lucide icons. Operates as a fast Single Page Application (SPA). Provides instant client-side input validation and dynamic UI re-rendering without page refreshes.

02

Application & API Tier (Node.js + Express.js)

Handles stateless RESTful endpoints, bcrypt password encryption (10 rounds), and JSON Web Token (JWT) verification. Implements strict Role-Based Access Control (Admin vs. Staff) and business logic.

03

Cloud Database Tier (MongoDB Atlas)

Hosted on a highly-available cloud replica set with automatic failover. Stores schema collections in native JSON/BSON format, utilizing compound indexes for sub-50ms query execution under heavy load.

TALKING POINT FOR PRESENTATION: "Explain that we chose the MERN stack because JavaScript runs seamlessly from browser to database. Point to the 3 tiers: React gives staff a snappy interface, Express enforces authentication and business rules, and MongoDB Atlas manages data safely in the cloud."

MongoDB Schema Architecture & Design Rationale

Key Architectural Decision: Referencing vs. Embedding

In MongoDB, embedding transaction logs inside a product document would cause the document to breach the 16 MB limit within months in an active warehouse. Therefore, we used normalized referencing with ObjectId links for StockMovements. This guarantees infinite historical audit logging while keeping product queries ultra-fast.

Users Collection

- Stores username, email, hashed password, and role ('admin' vs 'staff').
- Uses bcrypt with 10 salt rounds.
- Enforces role-based permissions so only admins manage warehouses/suppliers.

Products Collection

- Stores SKU, name, category, unit price, quantity, and minStockThreshold.
- Indexed on SKU (unique) for instant lookup.
- Supports soft-delete ('isActive') so past movement histories are never corrupted.

Stock Movements

- Immutable audit log for every transaction.
- Records type ('IN', 'OUT', 'TRANSFER'), quantity, previous vs new stock, reference, and date.
- Links to Product and User via ObjectId references.

Warehouses & Suppliers

- Warehouses: Multi-location tracking with physical address, capacity, and supervisor.
- Suppliers: Contact name, lead time, and payment terms.
- Keeps supplier-product supply chains organized.

TALKING POINT FOR PRESENTATION: "Highlight our Referencing design decision. Evaluators love database schema justification! Explain that referencing prevents hitting MongoDB's 16 MB document size ceiling and allows unlimited historical audit logging without slowing down daily queries."

Core Working Principle: 5-Step Transaction Lifecycle

Step 1
User Action

01

Staff initiates a Stock IN or OUT form, selecting the target product, warehouse, quantity, and reference invoice note.

Step 2
Client Validation

02

React verifies positive integer inputs and checks requested dispatch quantity against current physical balance before submitting.

Step 3
API Authentication

03

Express verifies the user's JWT token, validates role permissions, and starts an atomic controller transaction.

Step 4
Atomic Database Update

04

MongoDB executes a conditional update ($inc) ensuring stock cannot drop below zero. If condition fails, transaction aborts.

Step 5
Audit Log & Sync

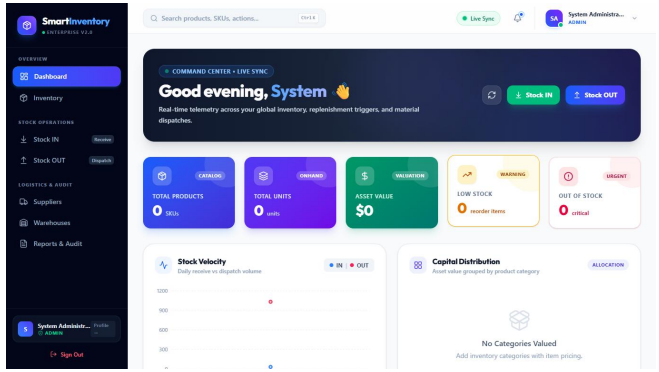
05

An unalterable StockMovement record is written, and dashboard KPI cards update in real time without refreshing.

TALKING POINT FOR PRESENTATION: "This is your best demonstration slide! Walk the evaluators step-by-step from Step 1 to Step 5. Point out that a stock change is never just an edit—it is validated twice, updated atomically in MongoDB, and logged permanently in the movement journal."

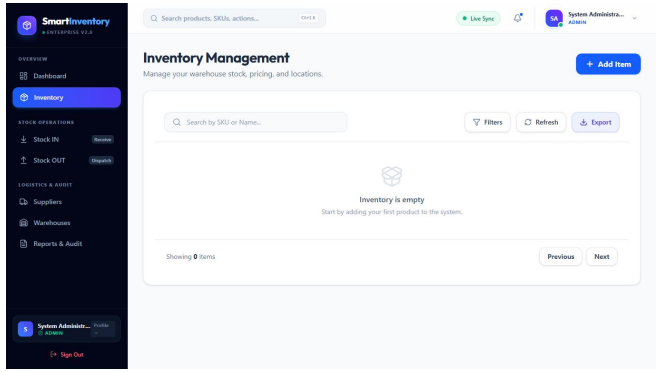
Live System Walkthrough: Production Interfaces

Operational Dashboard



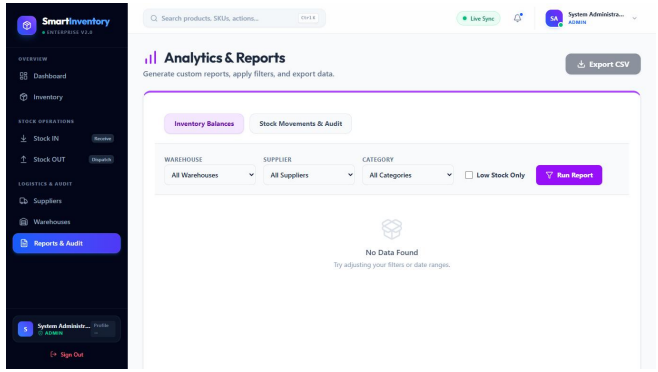
- Top-level KPI summary cards: Total Items, Inventory Valuation, Low-Stock Alerts, and Out-of-Stock warnings.
- Quick-action buttons allow staff to log Stock IN / OUT in under 5 seconds.
- Gives warehouse supervisors a real-time operational pulse upon login.

Real-Time Inventory Catalog



- Filterable and searchable catalog with live SKU search, category pills, and warehouse location badges.
- Dynamic color indicators instantly show items nearing depletion.
- Supports inline edit, restocking, and soft-delete archive.

Comprehensive Movement Reports



- Complete chronological transaction audit log with filter by date, warehouse, and movement type.
- Displays exact quantities, timestamp, and authenticated user identity.
- Eliminates disputes by providing proof of every dispatch.

TALKING POINT FOR PRESENTATION: "Point to the 3 actual screens of our running application: The Dashboard gives managers the big picture, the Inventory Catalog lets clerks find and restock items quickly, and the Reports tab provides an undeniable audit history."

Key Engineering Challenges & Technical Solutions

01

Preventing Race Conditions & Negative Stock

The Hurdle: In high-traffic warehouses, two clerks dispatching the last 5 items at the exact same moment could result in a negative balance of -5, breaking accounting integrity. **How We Solved It:** We implemented atomic conditional updates at the database level using Mongoose query conditions: { _id: id, quantity: { $gte: reqQty } }, combined with schema-level validation min: 0. If stock is insufficient, the operation is blocked atomically.

02

Node.js 24 Cloud DNS Resolution on Atlas

The Hurdle: Node.js v24 introduced strict RFC-compliant DNS SRV lookups, causing intermittent connection timeouts when resolving mongodb+srv:// Atlas URIs on certain ISP routers. **How We Solved It:** We configured explicit public DNS resolvers in server.js using dns.setServers(['8.8.8.8', '8.8.4.4']), ensuring instantaneous, reliable database connection in all local and cloud deployment environments.

03

Real-Time UI Synchronization Without Reloads

The Hurdle: Staff shouldn't have to hit browser refresh (F5) to see updated quantities or low-stock banners after completing a dispatch or receipt. **How We Solved It:** We engineered optimistic UI updates and reactive state hooks in React. The moment the backend confirms the stock change, local state updates seamlessly and triggers an async metric re-fetch in the background.

TALKING POINT FOR PRESENTATION: "Evaluators love seeing technical problem-solving! Explain that we didn't just write a simple CRUD app—we solved real concurrency race conditions, resolved Node 24 DNS lookups on MongoDB Atlas, and built reactive state sync."

Comparative Analysis: Traditional vs. SmartInventory

Traditional Spreadsheets & Paper

- Prone to human typo errors and accidental formula overwrites.
- Zero concurrency protection when multiple staff members log dispatches.
- No audit trail; impossible to prove who removed stock or when.
- Reordering is purely reactive after products run out completely.
- Requires tedious manual stock counts that take hours each week.

SmartInventory Web Platform

- 1-Click stock adjustments with automated mathematical precision.
- Atomic database writes guarantee consistent inventory counts.
- 100% immutable transaction journal with user ID and timestamps.
- Proactive low-stock warnings alert managers before stockouts occur.
- Real-time cloud dashboard eliminates manual stock reconciliations.

TALKING POINT FOR PRESENTATION: "Directly contrast the two sides for the committee. On the left is the messy, error-prone spreadsheet reality that causes warehouse losses. On the right is SmartInventory: instant, atomic, fully auditable, and proactive."

Future Roadmap & Scalability Enhancements

PLANNED ENGINEERING ROADMAP: 4 STRATEGIC PHASES

Phase 1

Barcode & QR Scanning

Direct mobile camera barcode scanning integrated into the browser, enabling storekeepers to scan boxes for instant Stock IN / OUT without typing SKUs manually.

Phase 2

AI Demand Forecasting

Machine learning regression models analyzing seasonal purchase trends to recommend automated reorder quantities and prevent over-stocking dead capital.

Phase 3

Automated Supplier POs

Automated purchase order email generation dispatched directly to registered suppliers the second an item drops below safety threshold.

Phase 4

Offline PWA Sync

Progressive Web App with local IndexedDB storage, enabling warehouse workers in zero-connectivity basement racks to log movements and sync automatically once online.

TALKING POINT FOR PRESENTATION: "Show the committee that you have a clear vision for real-world scaling. Highlight barcode scanning for speed, AI forecasting to reduce dead stock, automated supplier emails, and offline PWA mode for basement warehouses."

Project Summary

✓ **100% Pass Rate**
Verified across 8 formal functional test cases (TC-01 through TC-08).

✓ **Zero Negative Stock**
Enforced via dual client-side and backend atomic guards.

✓ **Sub-50ms Queries**
Optimized with MongoDB compound indexes and referencing schema.

✓ **Complete Audit Trail**
Every transaction logged permanently with user attribution.

Thank You!

Questions & Technical Discussion

SmartInventory demonstrates how modern full-stack web technologies and cloud document databases can transform manual warehouse operations into a fast, accurate, and completely transparent workflow.

- **Live Application & Source Code:** https://github.com/ShubhamKumar284/Inventory-Management-System
- **Technical Project Report:** Complete 15-page academic documentation available for review.

TALKING POINT FOR PRESENTATION: "Conclude by thanking the faculty guide and evaluation committee. State that you are prepared to demonstrate any live feature or dive deeper into the MongoDB queries and backend code."